

COMSATS University Islamabad, Abbottabad
Campus



Lab Task #03

Submitted By:

Umer Saleem: SP24-BSE-171

Class:

BSE-6A

Subject:

Software Quality Engineering

Submitted To:

Sir Mukhtiar Zamin

Date:

21-Sep-2026

Activity 1 — (Review of FixKar SRS)

1. Ambiguities:

Ambiguity means a requirement is **not clear enough and can be understood in more than one way**.

Ambiguity 1 — Registration Method

Requirement: The system allows registration using a mobile number **or** email.

Problem: It is not clear how OTP verification will work for a user who registers only with an email address, because the SRS specifically mentions SMS OTP.

Suggestion: Clearly define whether email users will receive an email OTP or whether a mobile number is compulsory.

Ambiguity 2 — Cancellation Policy

Requirement: Customers can cancel bookings according to a cancellation policy.

Problem: The exact cancellation rules, fee amount, and time limits are not clearly defined.

Suggestion: Clearly specify the cancellation fee and the time period in which cancellation is free or charged.

Ambiguity 3 — Payment Gateway

Requirement: The system will use a licensed third-party payment gateway such as JazzCash, EasyPaisa, or Stripe.

Problem: The exact payment gateway to be used is not selected.

Suggestion: Specify the final payment gateway and its supported payment methods.

Ambiguity 4 — “Near Real Time”

Requirement: Dashboard KPIs should refresh in near real time.

Problem: “Near real time” is not a precise time.

Suggestion: Define an exact time, such as **within 2 seconds**, for KPI updates.

2. Missing Requirements:

Missing requirements are important functions or details that are **not clearly specified in the SRS**.

Missing Requirement 1 — Exact Cancellation Rules

The SRS mentions a cancellation policy but does not clearly define all cancellation conditions and charges.

Required: Define cancellation time limits, fees, and refund rules.

Missing Requirement 2 — Refund Process

The SRS explains successful and failed payments but does not clearly describe how a customer receives a refund after a cancelled or disputed payment.

Required: Add requirements for refund processing and refund status.

Missing Requirement 3 — Chat Requirements

The SRS allows Customer and Service Provider communication through chat, but it does not clearly define whether chat messages can contain images, how long messages are stored, or how inappropriate messages are handled.

Required: Define basic chat rules and message handling.

Missing Requirement 4 — Provider Availability Details

The SRS allows providers to set weekly availability, but it does not clearly explain how holidays, temporary unavailability, or overlapping bookings are handled.

Required: Define complete provider availability rules.

Missing Requirement 5 — Account Deletion Data Handling

The SRS allows users to permanently delete their accounts, but the exact data-retention period and which records remain after deletion are not clearly specified.

Required: Define what data is deleted and what data is retained for legal/audit purposes.

3. Inconsistencies:

Inconsistency means that **two parts of the SRS do not completely match each other.**

Inconsistency 1 — Booking Status

One requirement lists the booking statuses as:

Requested → Accepted → In Progress → Completed → Cancelled

But another part of the SRS also defines an **Expired** state when no provider accepts within the response window.

Problem: Expired is missing from the status list in the Customer Module.

Correction: Include **Expired** in all booking-status definitions.

Inconsistency 2 — Provider Job Status

The Service Provider Module defines these statuses:

En-route → Arrived → In Progress → Completed

However, the booking lifecycle mainly shows:

Accepted → In Progress → Completed

Problem: En-route and Arrived are not consistently represented in the booking lifecycle.

Correction: Clearly define whether these are separate booking states or only provider-specific job statuses.

Inconsistency 3 — Android and iOS Support

The SRS says the mobile application is designed so that it can later be extended to iOS, while another requirement states that the **initial release targets Android only**.

Problem: The document does not clearly distinguish current support from future support in every section.

Correction: Clearly state: “**Version 1.0 supports Android only; iOS support is planned for a future version.**”

Inconsistency 4 — Payment Timing

The Payment Module states that payment is made for a **completed booking**, while the system also includes online payment as part of the booking flow.

Problem: It is not completely clear whether online payment happens before the service, after completion, or both depending on the payment method.

Correction: Clearly define the payment timing for **Card, Wallet, and Cash**.

Final Conclusion:

The FixKar SRS contains many detailed requirements, but some requirements can still be improved. The main issues found are **unclear registration and cancellation rules, missing refund and availability details, and inconsistencies in booking statuses and payment timing**. These issues should be clarified so that developers, testers, and users can understand the requirements in the same way.

Activity 2 — (CMMI Level 1 Scenario Report)

Scenario:

Student Grading System:

A small software company is developing a student grading system. The team works quickly to meet deadlines but faces frequent issues.

Task 1:

Identify Level 1 Processes:

Question:

Identify at least four Level 1 processes present in the scenario.

Solution:

1. **Directly updating the live system when bugs are reported.**
2. **Requirements are discussed in meetings but are not formally documented.**
3. **Testing is performed only after customers complain about errors.**
4. **Each developer uses their own coding style and folder structure.**
5. **Defects are tracked informally through emails without a central log.**

These are the Level 1 processes found in the given scenario.

Task 2:

Explain Why Each Process Is Level 1:

Question:

Explain why each process reflects **chaotic, reactive, or non-standardized** practices.

Solution

1. Directly updating the live system:

This is **chaotic and reactive** because developers fix bugs directly in the live system without following a proper testing or release process.

2. Requirements are not documented:

This is **non-standardized** because requirements are only discussed in meetings. There is no proper document that everyone can follow.

3. Testing after customer complaints:

This is **reactive** because the team starts testing only after customers report errors instead of testing before releasing the system.

4. Different coding styles and folder structures:

This is **non-standardized** because every developer follows their own coding style and folder structure instead of using one common standard.

5. Defects tracked through emails:

This is **chaotic and informal** because there is no central defect tracking system. Important defects can be missed or difficult to manage.

Task 3:

Suggest QA Improvements:

Question:

Suggest one QA improvement for each process that could help move the company toward **Level 2 (Managed)**.

Solution:

1. Directly updating the live system:

QA Improvement:

Use a proper **testing and release process**. Every bug fix should be tested before it is moved to the live system.

2. Requirements are not documented:

QA Improvement:

Create a **requirements document** and review it before starting development.

3. Testing after customer complaints:

QA Improvement:

Prepare **test cases** and **perform testing before releasing** the software to customers.

4. Different coding styles and folder structures:

QA Improvement:

Create **common coding standards** and **folder structure** that all developers must follow.

5. Defects tracked through emails

QA Improvement:

Create a **central defect log** where all defects are recorded, assigned, and tracked until they are fixed.

Conclusion:

The student grading system has many **CMMI Level 1** problems because its processes are chaotic, reactive, and not standardized. The company can improve its quality by using **proper documentation, testing before release, coding standards, and a central defect log**. These simple QA practices can make the development process more organized and repeatable and help the company move toward **CMMI Level 2 (Managed)**.

Homework — (Multi-Level Scenario Analysis & Process Transformation)

Task 1: Defect & Process Identification

Scenario:

EduTech Solutions has project delays, post-release bugs, high employee turnover, and inconsistent code quality. The five observed practices can be mapped to Level 1 symptoms as follows.

1. Direct implementation of client requests — Reactive:

Developers implement new features immediately without checking their effect on existing code. This is reactive because they respond directly to the client's request without following a proper checking process.

2. "Gut feeling" project estimations — Chaotic:

Project managers estimate the time based on their personal guess instead of using previous project data or proper planning. This can cause delays and inaccurate schedules.

3. Inconsistent folder structures — Lack of Standardization:

Every developer uses their own folder structure and naming conventions. There is no common standard, making the code difficult to understand and maintain when a developer leaves.

4. Unvetted subcontractors — Chaotic:

Subcontractors are hired without proper quality checks. Their work may have quality problems because there is no formal process for checking their deliverables.

5. Ad hoc pre-release testing — Reactive:

Testing is performed only at the last moment before deployment. This means problems may be discovered too late instead of being identified earlier during development.

Task 2: Level 2 (Managed) Remediation Plan

The goal of Level 2 is to introduce **basic project-level management and repeatable practices**. The lab specifically suggests using REQM, PP, PMC, CM, PPQA, SAM, and MA for this transformation.

Issue #	Identified Failure	Proposed Level 2 Process Area	Concrete QA Mechanism
1	Direct implementation of client requests	REQM – Requirements Management	Use a Requirement Change Form and Traceability Matrix before implementing any new request.
2	"Gut feeling" project estimations	PP – Project Planning	Create a Project Plan with realistic schedules, resources, and estimated time based on previous project data.
3	Inconsistent folder structures and naming conventions	CM – Configuration Management	Use a standard folder structure, naming checklist, and version control system for all projects.
4	Unvetted subcontractors	SAM – Supplier Agreement Management	Use a Supplier Evaluation Checklist and quality requirements before hiring subcontractors.
5	Ad hoc pre-release testing	PPQA – Process and Product Quality Assurance	Create a testing checklist and QA review log that must be completed before deployment.

Short Conclusion

EduTech Solutions is showing several **Level 1 characteristics**, such as reactive work, poor planning, and lack of standardization. By introducing Level 2 practices such as **requirements management, project planning, configuration management, supplier management, and QA checks**, the company can make its projects more controlled and repeatable.
